
```

rm(list = ls())

# Local github
codepath <- c("/media/larryleon/My Projects/GitHub/Forest-Search/R/")
source(paste0(codepath, "source_forestsearch_v0.R"))
source_fs_functions(file_loc = codepath)

library(kableExtra)
library(knitr)
library(ggplot2)
library(gridExtra)
library(cubature)
library(aVirtualTwins)
library(randomForest)
library(survival)
library(survminer)
library(grf)
library(policytree)
library(data.table)
library(plyr)
library(dplyr)
library(glmnet)
library(corrplot)

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)
    stop("pH_super approximation relative error exceeds 10%")
  z1_frac <- z1_q
}

```

```

    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
t.start.all <- proc.time()[3]

##### Forest search criteria
hr.threshold <- 1.25 # Initial candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)
fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

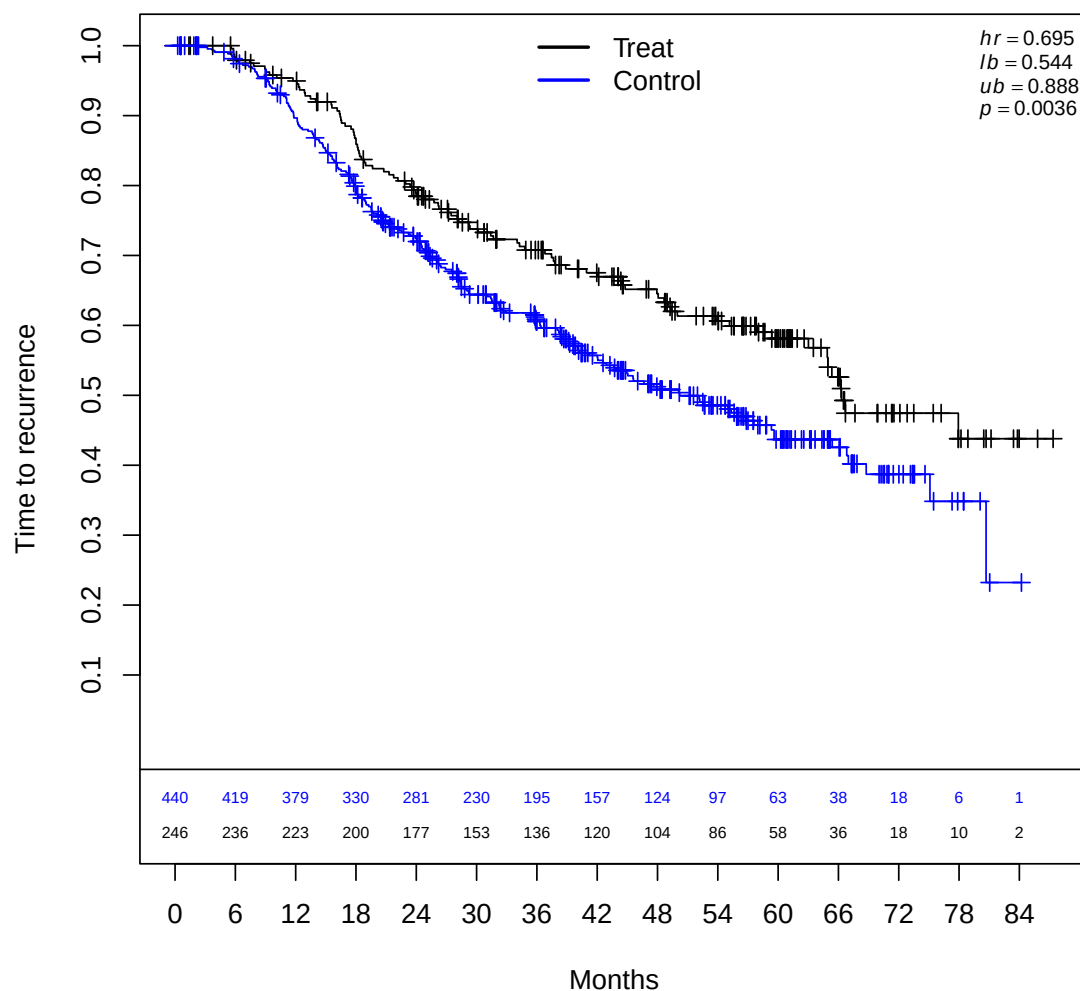
maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

mod.harm <- "alt"
hrH.target <- 2.5
# out.loc = NULL turns off file creation
N <- 700
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, hrH.target = hrH.target,
  cens.type = cens.type, out.loc = NULL, details = TRUE, parms_torand = FALSE)

```



```
## Super-population empirical harm and non-harm hazard ratios= 2.499996 0.6466405
## Causal HR (empirical ITT)= 0.7125876
```

```
file_out <- c("output/sim-FS4_Lasso_ex6.Rdata")

n_add_noise <- 3

# sim=69 sim=71

sim <- 41

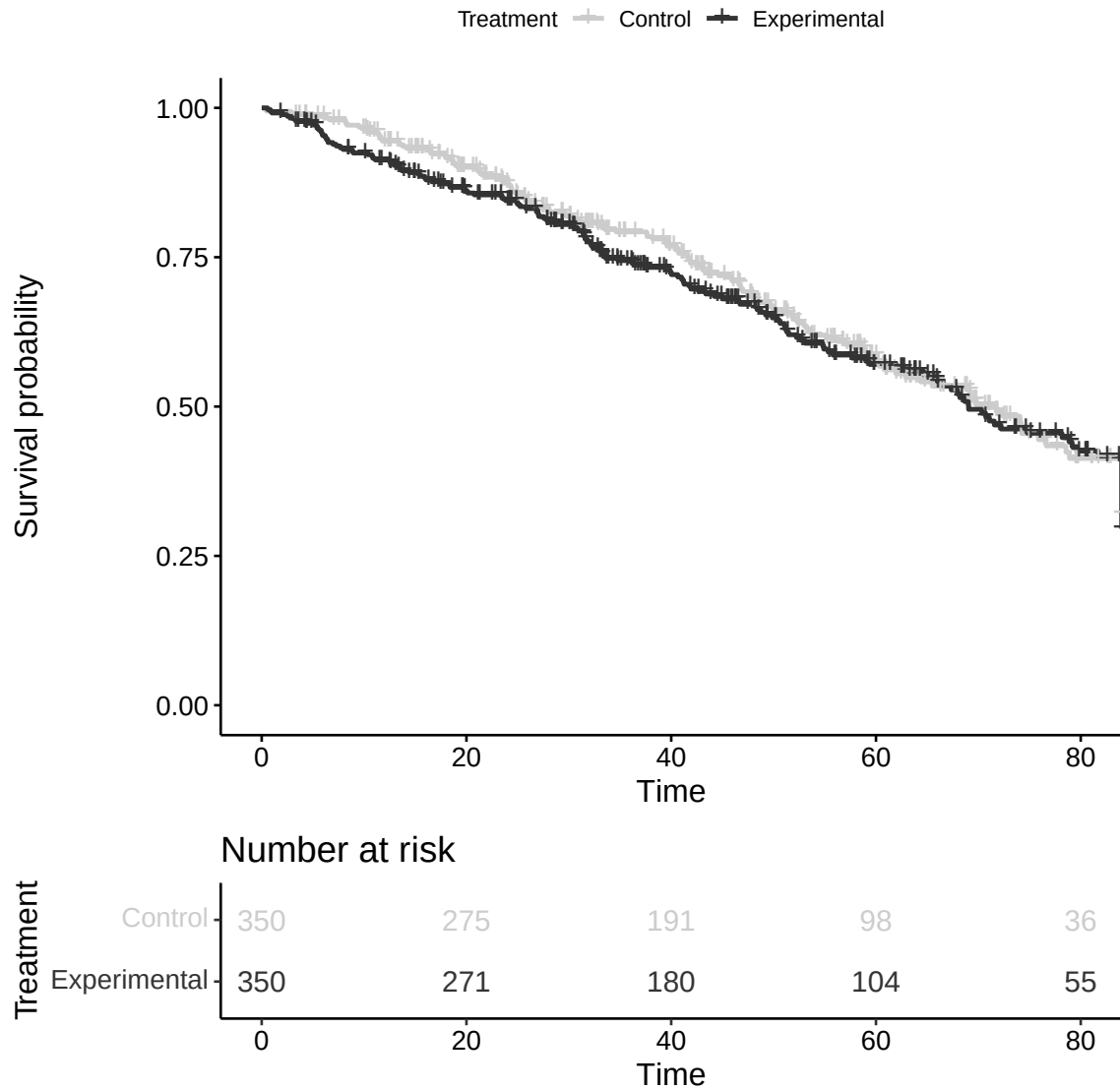
dgm <- this.dgm$dgm

x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim)

kmfit <- survfit(Surv(y.sim, event) ~ treat, data = x)

ggsurvplot(kmfit, data = x, main = "K-M curves for simulated data", legend = "top",
```

```
legend.title = "Treatment", legend.labs = c("Control", "Experimental"), palette = "grey",
risk.table = TRUE, risk.table.col = "strata")
```



```
if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade")
}

if (n_add_noise == 5) {
  set.seed(8316951 + 1000 * sim)
  # Add 5 noise
  x$noise1 <- rnorm(N)
  x$noise2 <- rnorm(N)
  x$noise3 <- rnorm(N)
  x$noise4 <- rnorm(N)
  x$noise5 <- rnorm(N)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade", "noise1",
    "noise2", "noise3", "noise4", "noise5")
}
```

```

}

if (n_add_noise == 3) {
  set.seed(8316951 + 1000 * sim)
  # Add 3 noise
  x$noise1 <- rnorm(N, sd = 1)
  x$noise2 <- rnorm(N, sd = 1)
  x$noise3 <- rnorm(N, sd = 1)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade", "noise1",
    "noise2", "noise3")
}

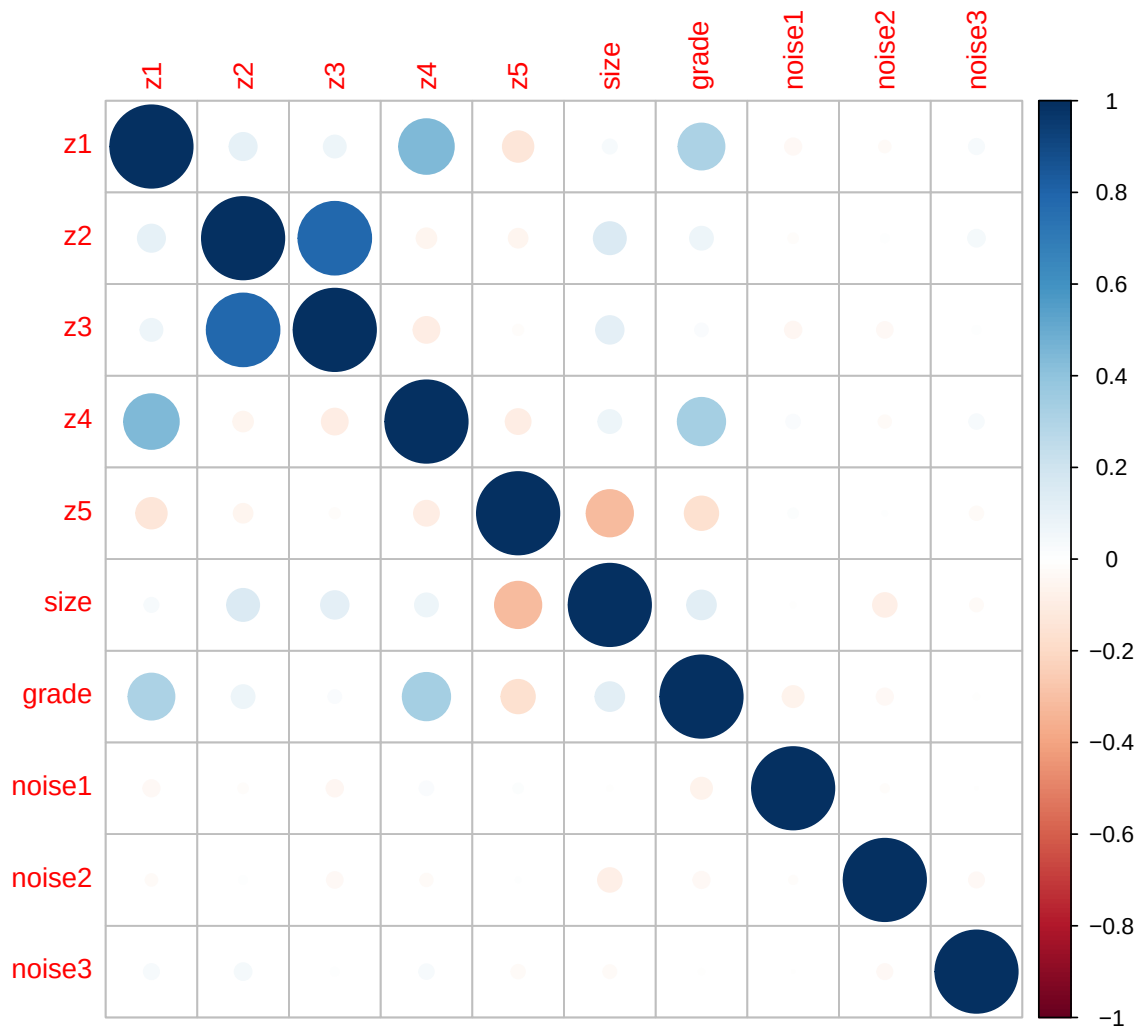
# More challenging which_replace <- which(confounders.name=='z1')
# confounders.name[which_replace] <- 'er'

cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5+size+grade+noise1+
coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef)    se(coef)      z        p
## treat  -0.2294158  0.7949979  0.1072809  -2.138  0.032479
## v11      0.4930358  1.6372791  0.1253887   3.932  8.42e-05
## v21     -0.2672528  0.7654796  0.1725905  -1.548  0.121507
## v31      0.1319377  1.1410373  0.1742385   0.757  0.448915
## v41      0.4711125  1.6017751  0.1248358   3.774  0.000161
## v51     -0.8279689  0.4369358  0.1129207  -7.332  2.26e-13
## size      0.0009854  1.0009859  0.0033421   0.295  0.768109
## grade     0.0123218  1.0123981  0.0985964   0.125  0.900545
## noise1 -0.0516051  0.9497038  0.0520261  -0.992  0.321243
## noise2  0.0442747  1.0452694  0.0497698   0.890  0.373686
## noise3 -0.0583977  0.9432747  0.0560947  -1.041  0.297850
##
## Likelihood ratio test=128.1 on 11 df, p=< 2.2e-16
## n= 700, number of events= 371

Zm <- cor(as.matrix(x[, c(confounders.name)]))
corrplot(Zm)

```



```
# Options Allconfounders.name is list of confounders within analysis dataset
# (1) use_lasso=TRUE & use_grf=FALSE Lasso used to possibly reduce dimension
# Any continuous factors are cut at medians (2) use_lasso=TRUE & use_grf=TRUE
# Lasso used to reduce dimension Continuous covariates are cut at medians
# However, if GRF selects a covariate cut then only that cut is used: For
# example if 'age <= median(ag)' is called for per Lasso but GRF includes 'age
# <= 54', then only the latter is used (3) use_grf_only = TRUE (overrides
# use_lasso and use_grf) Only factors selected via GRF are used (4) use_lasso =
# F & use_grf =T Median cuts (unless selected via GRF) as in (2) However no
# possible dimension reduction via lasso All categorical factors included

use_lasso <- TRUE
use_grf <- FALSE
use_grf_only <- FALSE

fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, use_grf_only = use_grf_only, dmin.grf = 12,
  frac.tau = 0.6, conf_force = NULL, outcome.name = outcome.name, treat.name = treat.name,
```

```

event.name = event.name, id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold,
hr.consistency = hr.consistency, fs.splits = fs.splits, d0.min = d.min, d1.min = d.min,
pstop_futile = pstop_futile, pconsistency.threshold = pconsistency.threshold,
stop.threshold = stop.threshold, max.minutes = max.minutes, maxk = maxk, by.risk = 12,
plot.sg = TRUE, vi.grf.min = vi.grf.min)

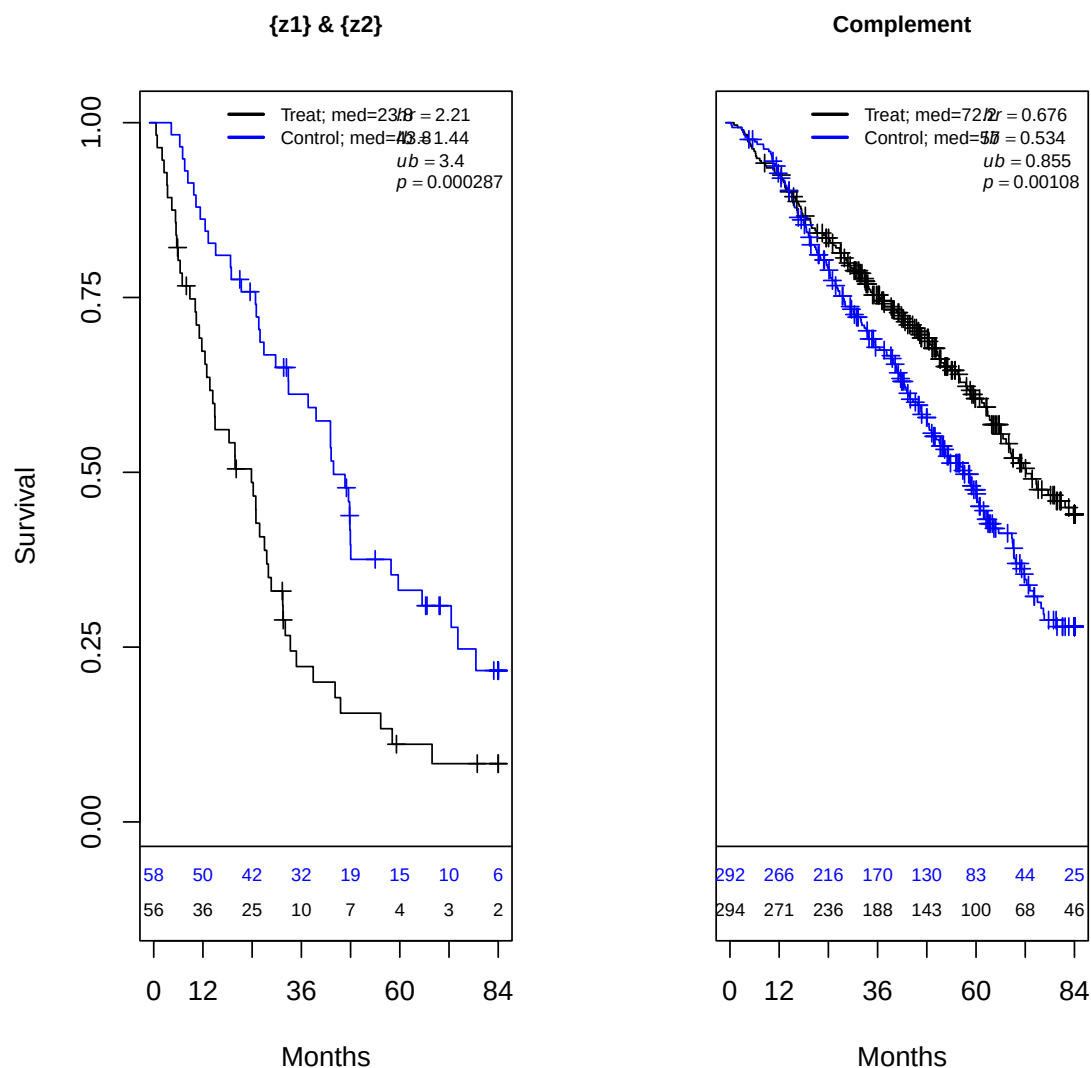
## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade
## 10 x 2 sparse Matrix of class "dgCMatrix"
##           s0      s0
## z1      0.48618275 0.63323036
## z2     -0.08454916 0.11012134
## z3      .          .
## z4      0.41902227 0.54575696
## z5     -0.76778181 1.00000000
## size      .          .
## grade      .          .
## noise1 -0.02414378 0.03144615
## noise2  0.02278763 0.02967982
## noise3 -0.03920336 0.05106055
## Cox-LASSO selected: z1 z2 z4 z5 noise3
## Cox-LASSO not selected: z3 size grade noise1 noise2
## Median cuts (after Lasso if use_lasso) noise3
## Categorical (after Lasso if use_lasso) z1 z2 z4 z5
## Prior to removing (if in GRF): noise3
## Left (to cut at median) after removing others if in GRF: noise3
## # of candidate subgroup factors= 5
## [1] "noise3 <= median(noise3)" "z1"
## [3] "z2"                        "z4"
## [5] "z5"
## Confounders per grf screening q2 q3 q5 q1 q4
## FSconfounders.name      vi.cs vi_ratio
## 2          q2 0.4054511 2.0272556
## 3          q3 0.2246710 1.1233551
## 5          q5 0.1370532 0.6852662
## 1          q1 0.1307915 0.6539575
## 4          q4 0.1020331 0.5101656
## Number of unique levels (L) and possible subgroups= 10 1023
## # of subgroups based on # variables > k.max and excluded (per million) 0.000968
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 6 6
## # of subgroups with sample size less than criteria 6
## # of subgroups meeting all criteria = 49
## # of subgroups fitted (Cox model estimable) = 49
## *Subgroup Searching Minutes=* 0.0021
## Number of subgroups meeting HR threshold 5
## # of candidate subgroups (meeting HR criteria) = 5
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##      K   n   E d1    m1    m0  HR L(HR) U(HR) q2.0 q2.1 q3.0 q3.1 q5.0 q5.1
## 1:  2 114  87 47 23.83 43.84 2.21  1.44  3.40   0   1   0   1   0   0
## 2:  2  93  69 35 21.13 43.65 1.66  1.03  2.68   0   1   0   0   0   0
## 3:  2 112  89 50 21.13 33.80 1.54  1.01  2.36   0   1   0   0   1   0
## 4:  1 188 132 68 28.65 43.84 1.36  0.96  1.91   0   1   0   0   0   0
## 5:  2 165 119 61 31.45 43.10 1.26  0.88  1.80   0   1   0   0   0   0

```

```

## 6: NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
## 7: NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
## 8: NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
## 9: NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
## 10: NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
##      q1.0 q1.1 q4.0 q4.1
## 1:      0      0      0      0
## 2:      0      1      0      0
## 3:      0      0      0      0
## 4:      0      0      0      0
## 5:      0      0      0      1
## 6:      NA      NA      NA      NA
## 7:      NA      NA      NA      NA
## 8:      NA      NA      NA      NA
## 9:      NA      NA      NA      NA
## 10:     NA      NA      NA      NA
## Consistency 1
## # of splits= 400
## Model, % Consistency Met= {z1} {z2} 1
## Number of subgroups meeting consistency criteria=
##      p.consistency Nsg group.id m.index K  M.1  M.2
## 1:                1 114          2      1 2 {z1} {z2}

```



```
## [1] "{z1}" "{z2}"
## % consistency criteria met= 1
## SG focus= hr
## Subgroup Consistency Minutes= 0.02326667
## Subgroup found
## Minutes overall= 0.0371

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 1, details = TRUE)

## tau, maxdepth= 79.59705 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2    -7.419304   512.000000   2.299844    7.419304   512.000000
## 2         3     6.746193   188.000000   3.779360   -6.746193   188.000000
## 11        4    -8.368215   498.000000   2.332510    8.368215   498.000000
## 3         6    -6.787540   101.000000   4.868356    6.787540   101.000000
## 4         7    22.457768    87.000000   5.456913   -22.457768    87.000000
##   treated.se      diff Nsg depth
```

```
## 1      2.299844 -14.83861 512      1
## 2      3.779360  13.49239 188      1
## 11     2.332510 -16.73643 498      2
## 3      4.868356 -13.57508 101      2
## 4      5.456913  44.91554  87      2
##      leaf.node control.mean control.size control.se treated.mean treated.size
## 4           7      22.457768      87.000000  5.456913  -22.457768      87.000000
##      treated.se      diff Nsg depth
## 4      5.456913 44.91554  87      2
## [1] "z1 <= 0"          "noise3 <= 1.82" "z3 <= 0"
## [1] "z3 <= 0"
```

```
library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 1000

df_boot_analysis <- fs.est$df.est

fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula)
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formula)
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_")

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")
## Minutes for Boots 1000 1.3055

cat("Projection per 1000", c(tB.min * (1000/NB)), "\n")
## Projection per 1000 1.3055

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")
```

```

## Propn bootstrap subgroups found = 0.946

# How many timed out
cat("Number timed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmins_search > max.minutes)),
    "\n")

## Number timed out= 0

H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)
Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)

print(H_estimates)

##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower H1_upper
## 1: 2.213426 0.484931 1.440717 3.400569 1.952362 0.5760672 1.094973 3.481107

print(Hc_estimates)

##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower H1_upper
## 1: 0.6755728 0.0810837 0.5339605 0.8547423 0.68618 0.119446 0.4878273 0.9651839

# H_estimates<-get_dfRes(Hobs=H_obs,seHobs=seH_obs,H1_adj=resB$H_biasadj_1,ystar=Ystar_mat,cov_metho
# Hc_estimates<-get_dfRes(Hobs=Hc_obs,seHobs=seHc_obs,H1_adj=resB$Hc_biasadj_1,ystar=Ystar_mat,cov_m
# print(H_estimates) print(Hc_estimates)

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
    sim_number = sim)
tall.min <- (tB.now - t.start.all)/60
cat("Overall minutes for analysis", c(tall.min), "\n")

## Overall minutes for analysis 1.431233

save(dgm, x, fs.est, grf.est, bootit, ans3, tall.min, resB, cox.formula.boot, file = file_out)

## Confounders evaluated in Forestsearch:
## [1] "noise3 <= median(noise3)" "z1"
## [3] "z2"                        "z4"
## [5] "z5"

```

Table 1 Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups H and H^c . Cox model estimates are based on subgroups: H true (knowing the actual subgroup, a-priori); the estimated subgroup $\hat{H}$; and the bootstrap (1,000 resamples) bias-correction to $\hat{H}$ estimates, denoted $\hat{H}_{bc}$. Estimates for the complement H^c are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups H and H^c , respectively, are listed.

	HR Estimate	Lower	Upper	# Subjects	# Correct
ITT					
ITT	0.827	0.674	1.014	700	.
H subgroup estimates					
H_{oracle}	3.076	1.839	5.144	87	.
$\hat{H}$	2.213	1.44	3.4	114	87
$\hat{H}_{bc}$	1.952	1.095	3.481	114	87
H-complement subgroup estimates					
H_{oracle}^c	0.686	0.547	0.86	613	.
$\hat{H}^c$	0.676	0.535	0.855	586	586
$\hat{H}_{bc}^c$	0.686	0.488	0.964	586	586